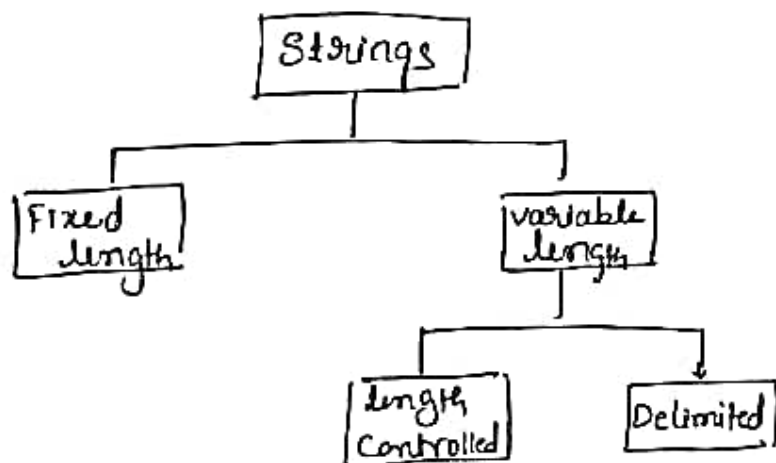


STRINGS:String taxonomy:

- The strings can store either in fixed length format or variable length format

Fixed length strings:

- The storing of string is a fixed length string, it needs to specify appropriate size for the variable.
- The size is too small, then will not be able to store all the elements in string.
- The string size is large, unnecessarily memory space will be wasted.

variable length string:

- The use of variable length format in which string can expand or contract to accommodate the element in it.
- Ex: String variable to store name of the student. If a student has a long name of 20 characters, then string can be expanded to accommodate 20 characters.
- The student name has only 5 characters then string variable can be contracted to store only 5 characters.
- To use a variable length string format, need a technique to indicate the end of elements that are part

of string.

→ It can be done either by using length-controlled string or a delimiter.

length controlled string:

→ length controlled string need to specify number of character in the string.

→ This count is used by string manipulation function to determine the actual length of string variable.

Delimited string:

→ The string is ended with delimiter.

→ The delimiter is used to identify the end of the string.

Ex: In English language every ~~element~~ sentence is ended with full stop (.) " " in 'C' we use any character such as comma, semicolon, colon, dash, null character etc as delimiter of string.

→ Null character is the most commonly used string delimiter in C language.

→ String is stored in an array of characters.

→ Use the null character as string delimiting character then we treat the part of the array from beginning to the null character as string & ignore rest.

```
char str[10];
```

```
gets(str);
```

If user enters HELLO.

H	E	L	L	O	\0				
---	---	---	---	---	----	--	--	--	--

part of array
not of string

Although array has '10' location only first '6' location will be treated as string.

Operations on strings:

- The different operations can be performed on character array before we start with these operations.
- Characters can be manipulated in same way as we do with numbers.
- we use character constants or character variable in an expression.
- It automatically converted into an integer value, where the value depends on local character set of system.

```
int i;
char ch='A';
i = ch;
printf("%d", i)
```

- 'C' enables programmers to perform arithmetic operations on character variables & character constants

```
int i;
char ch='A';
i = ch + 10;
printf("%d", i);
```

// prints 75;
ASCII value of ch 'A' is 'A'+10

- character variable and character constants can be used with relational operators

```
char ch='C';
if (ch >= 'A' && ch <= 'Z')
printf("The character is in upper case");
```

Finding the length of a string:

- The number of characters in the string constitutes the length of string

Ex: Length ("C programming is fun")

will return 19

Even blank spaces are counted as characters in the string.

length ("") = 0

length (" ") = 0

because both the strings do not contain any character $\therefore$ both strings are empty and of zero length.

→ Algorithm:

Step 1: [Initialize] SET $I = 0$

Step 2: Repeat step 3 while $STR[I] \neq \text{NULL}$

Step 3: SET $I = I + 1$
[End of loop]

Step 4: SET length = I

Step 5: END

→ 'I' is used as an index of the string STR. to traverse each & every character of STR we increment the value of I

→ Null character is encountered, control jumps out of while loop and length is initialized with value of I.

→ The number of character in the string constitutes its length.

write a program to find length of the string

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main ()
```

```
{
```

```
char str[100], i=0, length;
```

```
clrscr();
```

```
printf("In enter the string:");
```

```
gets(str);
```

```
while (str[i] != '\0').
```

```

i++;
length++;
printf("In length of the string is: %d", length);
getch();
return 0;
}

```

output:

Enter the string: HELLO
length of the string is: 5

Converting character of a string into upper case:

- In memory, ASCII codes are stored instead of actual value.
- ASCII code for A-Z varies from 65 to 90
ASCII code for a-z varies from 97 to 122
- If we have to convert lower case character to upper case then just need to subtract 32 from ASCII value of the character.

Algorithm:

- Step 1: [INITIALIZE] SET I = 0
- Step 2: Repeat Step 3 while STR[I] != NULL
- Step 3: IF STR[I] >= 'a' AND STR[I] <= 'z'
SET upperstr[I] = STR[I] - 32
ELSE
SET upperstr[I] = STR[I]
{END OF IF}
- Step 4: SET upperstr[I] = NULL
- Step 5: EXIT

- Initialize I to zero. Using I as index of STR, traverse each character Step 2 to 3.
- If character is already in upper case, then it is copied

in upperstr else the lower case character is converted ^(to upper)
upper case by Subtracting 32 from its ASCII value.

- upper case character is then stored in upperstr.
- All the characters have been traversed a null character is appended to upperstr. (step 4)

```
#include <stdio.h>
#include <conio.h>
int main()
{
    char str[100], upper-str[100];
    int i=0; j=0;
    clrscr();
    printf("\n enter the string:");
    gets(str);
    while (str[i] != '\0')
    {
        if (str[i] >= 'a' && str[i] <= 'z')
            upper-str[j] = str[i] - 32;
        else
            upper-str[j] = str[i];
        i++; j++;
    }
    upper-str[j] = '\0';
    printf("\n the string converted into upper case is:");
    puts(upper-str);
    return 0;
}
```

Output:

Enter the string: hello

String converted into upper case is: HELLO

Converting character of a string into lower case:

(7)

→ To convert an upper case character into lower case need to add 32 to its ASCII value.

Algorithm:

Step 1: [INITIALIZE] SET $i = 0$

Step 2: Repeat Step 3 while $STR[i] \neq NULL$

Step 3: IF $STR[i] \geq 'A'$ AND $STR[i] \leq 'Z'$

SET $Lowerstr[i] = STR[i] + 32$

ELSE

SET $Lowerstr[i] = STR[i]$

[END OF IF]

SET $i = i + 1$

[END OF LOOP]

Step 4: SET $Lowerstr[j] = NULL$

Step 5: EXIT

→ Initialize i to zero using i as the index of STR .

→ Traverse each character from Step 2 to 3

→ If the character is already in lower case, copied in $Lowerstr$ else upper case character is converted into lower case by adding 32 to its ASCII value.

→ Lowercase character is stored in $Lowerstr$.

→ When all character have been traversed a null character is appended to $Lowerstr$.

Write a program to convert character of a string into lowercase

```
#include <stdio.h>
```

```
#include <conio.h>
```

```
int main()
```

```
{
```

```
char Str[100], lower_str[100];
```

```
int i=0, j=0;
```

close();

printf("Enter the string:");

gets(str);

```
while (str[i] != '\0')
{
    if (str[i] >='A' && str[i] <='Z')
        lower-str[j] = str[i] + 32;
```

else

lower-str[j] = str[i];

i++; j++;

}

lower-str[j] = '\0';

printf("The string converted into lower case is:");

puts(lower-str);

return 0;

}

Output:

Enter the string: HELLO

String converted into lower
case is: hello

Concatenating two strings to form a new string:

→ The Concatenation operation produces a string which contains characters of S1 followed by characters of S2

Algorithm:

Step 1: [INITIALIZE] I=0 and J=0

Step 2: Repeat Steps 3 to 4 while str1[i] != NULL

Step 3: SET new-str[j] = str1[i]

Step 4: SET I = I+1 and J = J+1

[END OF LOOP]

Step 5: SET I=0

Step 6: Repeat Steps 6 to 7 while str2[i] != NULL

Step 7: SET new-str[j] = str2[i]

Step 8: SET I = I+1 and J = J+1

[END of loop]

Step 9: SET new-str[j] = NULL

Step 10: EXIT

~~char str3[100];~~ → Initialize the two counters i & j to zero.

~~printf~~ → To concatenate the strings, copy the contents of the first string followed by contents of second string in a third string, new-str.

→ Step 2 to 4 copies the contents of first string in new-str

→ Step 6 to 8 copies the contents of second string in new-str.

→ After the contents have been copied a null character is appended at the end of new-str.

Write a program to concatenate two strings:

#include <stdio.h>

#include <conio.h>

int main()

{

char str1[100], str2[100], str3[100];

int i=0, j=0;

clrscr();

printf("\n enter the first string:");

gets(str1);

printf("\n enter the second string:");

gets(str2);

while (str1[i] != '\0')

{

str3[j] = str1[i];

i++; j++;

}

i=0;

while (str2[i] != '\0')

{

str3[j] = str2[i];

i++; j++;

}

str3[j] = '\0';

printf("\n the concatenated string is:");

puts(str3);

} getch(); return 0;

Output:

Enter the first string: Hello

Enter the second string: How are you?

Concatenated string is:

Hello, how are you?

Appending a string to Another string:

- Appending one string to another string involves copying the contents of source string at the end of destination string.
- If S1 & S2 are two strings, then appending S1 to S2 means we have to add the contents of S1 to S2
- S1 is the source string and S2 is the destination string
- The append operation leave the source string S1 unchanged and destination string $S2 = S2 + S1$.

Algorithm:

- Step 1: [INITIALIZE] SET $I=0$ and $J=0$
- Step 2: Repeat Step 3 while $\text{Dest-Str}[I] \neq \text{NULL}$
- Step 3: SET $I = I + 1$
[END of loop]
- Step 4: Repeat Steps 5 to 7 while $\text{Source-Str}[J] \neq \text{NULL}$
- Step 5: $\text{Dest-Str}[I] = \text{Source-Str}[J]$
- Step 6: SET $I = I + 1$
- Step 7: SET $J = J + 1$ } [END OF LOOP]
- Step 8: SET $\text{Dest-Str}[I] = \text{NULL}$
- Step 9: EXIT

Write a program to append a string to another string:

```
#include <stdio.h>
#include <conio.h>
int main()
{
    char Dest-Str[100], Source-Str[50];
    int i=0, j=0;
    clrscr();
    printf("\n Enter the source string:");
    gets(Source-Str);
    printf("\n Enter the destination string:");
    gets(Dest-Str);
    while (Dest-Str[i] != '\0')
        i++;
    while (Source-Str[j] != '\0')
    {
```

11

```

    dest-Str[i] = Source-Str[j];
    i++; j++;
}

```

Dest-Str[i] = '\0';

Printf("\n after appending, the destination string is:");

puts(Dest-Str);

getch();

return 0;

}

Output :

Enter the Source string : How are you?

Enter the destination string : Hi

After appending, the destination string is : Hi, How are you?

Comparing two strings:

→ If S1 & S2 are two strings then comparing two strings will give either of these results.

a) S1 & S2 are equal

b) S1 > S2, when in dictionary order S1 will come after S2

c) S1 < S2, when in dictionary order S1 precedes S2

→ Compare two strings, each and every character is compared from both the strings.

→ All the characters are same then two strings are said to be equal

Algorithm:

Step 1: [INITIALIZE] SET I=0, SAME=0

Step 2: SET Len1 = Length(STR1), Len2 = Length(STR2)

Step 3: IF Len1 != Len2, then

write "Strings are not equal"

ELSE

Repeat while I < Len1

IF STR1[J] != STR2[I]

SET I = I+1

ELSE

Go to Step 4

[END of loop]

IF I = Len1

SET SAME = 1

write "Strings are equal"

[END of IF]

Step 4: IF SAME = 0

IF STR1[J] > STR2[J], then

write "String 1 is greater than String 2"

ELSE if STR1[J] < STR2[J], then

write "String 2 is greater than String 1"

[END of IF]

[END of IF]

Step 5: EXIT

write a program to compare two strings:

```
#include <stdio.h>
#include <conio.h>
#include <string.h>
int main()
{
    char str1[50], str2[50];
    int i=0, len1=0, len2=0, same=0;
    clrscr();
    printf("\nEnter the first string:");
    gets(str1);
    printf("\nEnter the second string:");
    gets(str2);
    len1 = strlen(str1);
    len2 = strlen(str2);
    if (len1 == len2)
    {
        while (i < len1)
        {
            if (str1[i] == str2[i])
                i++;
            else break;
        }
        if (i == len1)
        {
            same = 1;
            printf("\nTwo strings are equal");
        }
    }
}
```

```
}
}
if (len1 != len2)
    printf("\nTwo strings are not equal");
if (same == 0)
{
    if (str1[i] > str2[i])
        printf("\nString 1 is greater than String 2");
    else if (str1[i] < str2[i])
        printf("\nString 2 is greater than String 1");
}
getch();
return 0;
}
```

output:

Enter the first string: Hello
Enter the second string: Hello
Two strings are equal.